

Supplementary Material for Repo0

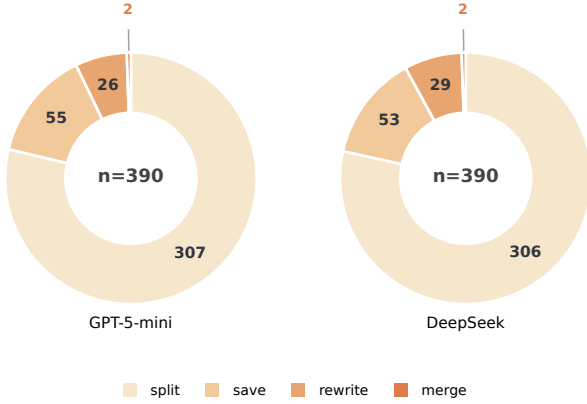


Fig. 1: Action distribution under a fixed GPT-5 mini first-round architecture. The figure compares subsequent structural evolution driven by GPT-5 mini and DeepSeek V3.2 on the same initial architecture.

A SUPPLEMENTARY RESULTS

This appendix reports the remaining repository-level results on the three repositories where RPG was not evaluated: *pandas*, *scikit-learn*, and *sympy*.

Figure 1 further compares the action distributions when DeepSeek V3.2 refines the same GPT-5 mini first-round architecture used by GPT-5 mini itself. The two distributions are highly similar: *save* remains dominant, *split* is consistently the second most frequent action, and the frequency of *revise* actions is especially close. This supports the interpretation in RQ3 that the cross-model difference in *revise* frequency mainly comes from the quality of the initial architecture rather than from substantially different refinement behaviors.

A. Repository-wise Cost Breakdown

Table II and Table III report the per-repository cost breakdown of Repo0 under DeepSeek V3.2 and GPT-5 mini, respectively. *Gen. Cost* denotes the cost of generating a repository from scratch with Repo0. *RPG Gen. Cost* reports RPG’s own repository-generation cost on the same repository when available; the parenthesized difference shows how much this cost is higher or lower than Repo0’s generation cost, with green indicating higher cost and red indicating lower cost. *Eval. Cost* denotes the cost of evaluating the generated repository using the RPG benchmark pipeline. *Total* is the sum of *Gen. Cost* and *Eval. Cost*.

Table II: Repository-wise cost breakdown of Repo0 under DeepSeek V3.2 (\$).

Repo	Gen. Cost	RPG Gen. Cost	Eval. Cost	Total
django	27.24	18.41 (-8.83)	72.82	100.06
pandas	11.02	—	54.05	65.07
requests	11.95	21.27 (+9.32)	9.33	21.28
scikit-learn	14.73	—	20.24	34.97
statsmodels	28.19	63.78 (+35.59)	12.93	41.12
sympy	21.24	—	11.91	33.15

Table III: Repository-wise cost breakdown of Repo0 under GPT-5 mini (\$).

Repo	Gen. Cost	RPG Gen. Cost	Eval. Cost	Total
django	24.08	40.76 (+16.68)	55.32	79.40
pandas	16.09	—	28.49	44.58
requests	8.91	36.20 (+27.29)	7.51	16.42
scikit-learn	23.14	—	21.30	44.44
statsmodels	73.33	92.97 (+19.64)	18.08	91.41
sympy	29.41	—	30.21	59.62

B. Requirement-Only Input Example

The following box shows the requirement-only input used for the RepoCraft counterpart of *requests*.

Requirement-Only Input Example for *requests*

HttpEasy: A Pythonic HTTP Library for Human Beings
 HttpEasy is an elegant and simple HTTP library for the Python programming language that brings simplicity and humanity to HTTP interactions. Designed from the ground up to prioritize developer experience and ease of use, HttpEasy has become one of the most widely adopted Python packages in the ecosystem, with approximately 30 million downloads per week and over 1 million repositories depending on it. The library serves as the de facto standard for HTTP communication in Python, enabling developers to interact seamlessly with web services, REST APIs, and remote HTTP endpoints without needing to grapple with the complexity of lower-level HTTP handling. At its core, HttpEasy abstracts away the intricacies of network programming while providing a clean, intuitive, and Pythonic interface that works for both simple one-off API calls and complex, long-lived sessions with persistent configuration. The library is built on top of urllib3 for the underlying transport layer, adding sophisticated session management, flexible authentication support, and a comprehensive suite of convenience features that make HTTP communication feel natural and effortless for Python developers. The library provides complete support for the entire HTTP/1.1 request-response cycle, enabling developers to construct HTTP requests using all standard methods (GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS) with minimal boilerplate, manage request parameters through query strings with automatic encoding, submit form data using application/x-www-form-urlencoded encoding, transmit structured data using JSON serialization directly within the library, and upload multipart files with support for customizable content types and headers. Beyond basic request construction, HttpEasy handles HTTP redirects transparently by automatically following Location headers with intelligent method conversion when appropriate and maintaining or stripping authentication credentials based on redirect destination to ensure security. The library implements persistent session management that maintains configuration, cookies, and pooled connections across

Table I: Supplementary RQ1 results on the three repositories where RPG is not evaluated: *pandas*, *scikit-learn*, and *sympy*. For each repository, we report Functionality Coverage (Cov.), Functionality Novelty (Nov.), and Pass./Vot., which combines Pass Rate and Voting Rate. **GPT-5 mini** marks the highest value among methods under GPT-5 mini, and **DeepSeek V3.2** marks the highest value among methods under DeepSeek V3.2.

Model	Method	pandas			scikit-learn			sympy		
		Cov. (%)	Nov. (%)	Pass./Vot. (%)	Cov. (%)	Nov. (%)	Pass./Vot. (%)	Cov. (%)	Nov. (%)	Pass./Vot. (%)
GPT-5 mini	mini-SWE-agent	47.37	1.67	1.57 / 34.51	61.70	3.17	5.00 / 6.00	18.75	22.54	3.08 / 26.15
	Paper2Code	59.21	20.15	27.84 / 50.20	21.28	14.96	14.00 / 20.00	50.00	21.88	26.15 / 72.31
	Repo0	71.05	16.88	72.55 / 97.65	80.85	12.23	79.00 / 92.00	81.30	10.30	43.30 / 63.20
DeepSeek V3.2	mini-SWE-agent	60.53	29.47	7.45 / 52.94	44.68	35.78	6.00 / 35.00	45.83	27.73	2.31 / 44.62
	Paper2Code	26.32	16.33	4.31 / 51.76	40.43	39.34	12.00 / 40.00	29.17	13.53	6.15 / 52.31
	Repo0	64.47	17.70	78.82 / 94.12	97.87	13.56	35.00 / 83.00	77.08	13.03	53.08 / 88.46
Human Developer	Gold Project	100.00	–	95.09 / 99.10	100.00	–	93.06 / 94.00	100.00	–	93.57 / 92.00

multiple requests, providing significant performance benefits for making many requests to the same host. Sessions support browser-style TLS/SSL verification with custom CA bundle support and client certificate authentication, implement both HTTP Basic Authentication and Digest Authentication schemes with challenge-response mechanisms including MD5 and SHA-256 digest algorithms and stale nonce handling, and allow developers to create custom authentication handlers by implementing callable objects that modify requests before transmission. HttpEasy also integrates seamlessly with network environments by supporting HTTP and HTTPS proxies through configuration options, handling SOCKS proxies for protocols beyond HTTP, automatically detecting proxy settings from environment variables with support for NO_PROXY configuration, and managing the .netrc configuration file for automatic authentication credential extraction. The library provides robust error handling through a comprehensive exception hierarchy that distinguishes between connection failures, timeout conditions, SSL/TLS certificate failures, proxy errors, HTTP protocol violations, and JSON parsing failures, allowing developers to implement sophisticated error recovery strategies tailored to specific failure modes. Advanced features include streaming responses for processing large response bodies without loading them entirely into memory, automatic content decompression for gzip and deflate encoded responses, intelligent character encoding detection and automatic decoding of response bodies in the appropriate character set, handling of chunked transfer encoding for both requests and responses, case-insensitive header handling for robust header access, request/response hooks that enable extending functionality at critical points in the request lifecycle through callback functions, connection pooling and keep-alive connections for improved performance when making multiple requests, fine-grained timeout control with separate settings for connection establishment and response data reading, status code reference mappings for convenient status code checking, and comprehensive cookie management with domain-aware persistence using a dict-like interface for cookie access. The implementation maintains compatibility with both CPython and PyPy interpreters while supporting Python 3.8 and later versions, ensuring broad compatibility across the Python ecosystem.

HttpEasy delivers a comprehensive HTTP communication framework that abstracts complexity while providing developers with an intuitive, Pythonic interface for every aspect of HTTP interaction, from simple GET requests to complex API workflows involving authentication, redirects, sessions, and error handling. The library orchestrates multiple components working in harmony: the convenient API functions provide entry points for quick requests, the Session class manages persistent state and configuration, Request and PreparedRequest objects encapsulate request intent and finalized form, HTTPAdapter handles the underlying urllib3 connection pools and protocol details, Response objects wrap server replies with automatic decoding, and a comprehensive exception hierarchy maps network and protocol errors to appropriate exceptions for consistent error handling. By elegantly combining these components, HttpEasy provides a complete solution for modern web API consumption in Python applications, enabling developers to focus

on their application logic rather than the mechanics of HTTP communication. The library’s philosophy emphasizes human-centric design, transparency in error handling, sensible defaults that work for most use cases, and the ability to customize behavior when needed, making it equally suitable for quick scripts and production-grade applications requiring reliability and robustness.

B PROMPT TEMPLATES

This appendix reports source-aligned prompt excerpts used by the continuous decision-driven structural evolution framework. We preserve the original prompt sections, instruction styles, and output schemas, while replacing repository-specific runtime payloads with placeholders.

A. Requirement Extraction

The requirement extraction stage turns a README or project document into a capability-level requirement set, filters out setup-only material, and merges semantically redundant requirements before dependency construction.

Requirement Extraction Prompt

You are a Senior Technical Product Manager and Business Analyst. Your task is to analyze the following project documentation and extract all functional requirements.

CLASSIFICATION

Requirements: The system must include a set of functionalities that are essential for performing its core tasks, while also optimizing performance, enhancing user experience, and improving overall efficiency. These functionalities work together to ensure the system operates effectively, reliably, and meets its intended objectives.

INSTRUCTIONS

1. Infer requirements from project documentation, dependencies, features, and implementation details. EXCLUDE all environmental, setup, and general configuration settings.
2. Strictly output a single, valid JSON object that adheres precisely to the required JSON schema.
3. Based on the previously extracted requirements, attempt to identify additional requirements that were not captured in previous rounds.

HIERARCHICAL REQUIREMENT STRUCTURE

- Extract requirements at an appropriate level: major capabilities should be separate requirements.
- For each major capability, if it has multiple related operations or features, include them comprehensively in the description field.

- Do not split low-level operations into separate requirements.
- The description should capture implementation details and sub-features of that capability.

INPUT PLACEHOLDERS

```
[Project Documentation]: {documentation_content}
PREVIOUSLY EXTRACTED REQUIREMENTS:
{requirements_list}
```

REQUIRED JSON SCHEMA

```
{
  "project_summary": "A concise summary of the project's purpose.",
  "requirements": [
    {
      "name": "string",
      "description": "string"
    }
  ]
}
```

Requirement Deduplication Prompt

You are a Senior Product Analyst specializing in requirements engineering and deduplication.

TASK

Analyze the provided list of requirements and identify items that are functionally redundant or significantly overlapping. Merge them into a single, comprehensive requirement.

INPUT DATA (JSON)

```
{requirements_payload}
```

MERGING CRITERIA

1. Merge when two or more requirements describe the same user goal or system behavior using different wording.
2. Merge when one requirement is clearly subsumed by another.
3. Keep requirements separate when they are merely related but still represent distinct responsibilities.
4. If overlapping requirements conflict, prioritize the most detailed and technically feasible description.

OUTPUT REQUIREMENTS

Return only a valid JSON object. If no redundancies are found, return `{"merge_groups": []}`.

RESPONSE SCHEMA

```
{
  "merge_groups": [
    {
      "reasoning": "...",
      "merged_name": "...",
      "merged_description": "...",
      "source_ids": [1, 2]
    }
  ]
}
```

RULES

- Each original id can belong to at most one merge group.
- Each `source_ids` list must contain at least two ids.
- Keep `merged_name` and `merged_description` in the same language as the input.

B. Requirement-Level DAG Construction

The requirement-level DAG prompt asks the model to emit requirement-level functional coordination relations that organize capabilities before component planning.

Requirement-Level DAG Construction Prompt

You are an expert Software Architect. Your task is to analyze project requirements and construct a directed acyclic graph (DAG) that captures *requirement-level functional coordination relations* for architectural planning.

CORE COORDINATION ANALYSIS FRAMEWORK

1. Prerequisite Relation: create edge $A \rightarrow B$ when capability A should be considered before capability B in the requirement space,

such as when user-facing behavior in B presupposes concepts, data, or interfaces introduced by A .

2. Support Relation: create edge $A \rightarrow B$ when capability A provides shared behavior, data abstractions, or interface expectations that help define capability B .

3. Shared Functional Concern: create edge $A \rightarrow B$ only when two capabilities should be decomposed, planned, or validated together because they belong to the same functional concern.

CRITICAL FILTERING RULES

- EXCLUDE testing requirements, documentation requirements, and auxiliary development tooling unless they are core to the product.
- EXCLUDE implementation-only relations, such as import, call, inheritance, or file-level containment, because these belong to the component-level DAG.
- EXCLUDE weak topical similarity that does not affect decomposition, planning, or validation.

COORDINATION CREATION BEST PRACTICES

- Create edge $A \rightarrow B$ only when A provides prerequisite or support context for understanding, decomposing, planning, or validating B .
- Ask: "Should these two capabilities be considered together before component-level implementation dependencies are known?" If no, do not create the edge.
- Keep only the most direct coordination relation and avoid transitive edges.
- Use exact requirement names from input; when in doubt, prefer fewer edges.

OUTPUT REQUIREMENTS

1. Output only a valid JSON object, with no explanations or markdown.
2. Format: `{"Parent Name": "Child Name"}`.
3. If no valid edges exist after filtering, return `{}`.
4. Both parent and child must be exact requirement names from the input list.

INPUT PLACEHOLDER

```
Input Project Requirements List:
{requirements_list}
```

C. Decomposition and Component Planning

The decomposition stage first splits a high-level requirement into a small set of implementable sub-requirements, then generates a cohesive component plan.

Sub-Requirement Decomposition Prompt

You are a software architect expert. Decompose the following high-level requirement into at most `{max_sub_requirements}` concrete, actionable sub-requirements.

```
Requirement Name: {requirement.name}
Description: {requirement.description}
```

CRITICAL RULES

1. Aim for two to three sub-requirements maximum; split only if truly necessary.
2. Each sub-requirement should be substantial and independently implementable.
3. Avoid over-decomposition; merge related tasks into a single sub-requirement when possible.
4. Skip trivial documentation and testing subtasks; focus on core implementation only.

FOR EACH SUB-REQUIREMENT, PROVIDE

- `name`: a short unique identifier.
- `description`: a clear, actionable implementation target.
- `order`: a zero-based sequential execution order.

OUTPUT FORMAT

Return only a JSON array of sub-requirement objects. Keep names concise; they will be prefixed with the parent requirement name automatically.

Architecture Planning Prompt

You are a software architecture expert. Design a unified, cohesive architecture for the following parent requirement and ALL its sub-requirements.

```
PARENT REQUIREMENT: {parent_name}
Description: {parent_desc}
SUB-REQUIREMENTS TO IMPLEMENT: {sub_reqs_text}
Environment Context:
{environment_feedback}
{dag_context}
```

CRITICAL INSTRUCTIONS

1. Prefer two to four components total; exceed four only when there is a clear, unavoidable architectural boundary.
2. Each component should serve multiple related sub-requirements; avoid one-to-one mapping between sub-requirements and components.
3. Optimize for the smallest set of high-cohesion, clearly ownable modules, not the longest feature checklist.
4. Merge adjacent responsibilities when they share the same data model, lifecycle, owner, and change together in practice.
5. Do not create separate components for validation, orchestration, export, adapters, helpers, or metadata unless they have clear standalone reuse value.
6. Split components only when there is strong evidence of different interfaces, dependencies, runtime constraints, or independent evolution.
7. Preserve domain semantics and avoid overly generic component names.

OUTPUT FORMAT

Return only a JSON object with `components`, `component_count`, and `rationale`. Each component includes `name`, `responsibilities`, and `serves_subrequirements`.

D. Component Dependencies and Structural Actions

The implementation-level dependency prompt infers must-have component dependencies. The refinement prompts then evolve the component-level structure through `save`, `split`, `revise`, `merge`, `add`, and `delete`, while using the requirement DAG as semantic context rather than as the direct action target.

Component Dependency Inference Prompt

You are analyzing implementation dependencies between planned software components.

GOAL

Infer **MUST-HAVE** implementation dependencies only. An edge `source → target` means the source component cannot be implemented correctly without importing, invoking, consuming, or structurally depending on the target.

IMPORTANT RULES

1. Cross-requirement edges are allowed and expected when implementation reuse is necessary.
2. Same-requirement edges are also valuable, but return them in a separate list.
3. The requirement DAG edges are soft hints, not hard constraints.
4. Prefer omission over weak guesses.
5. Do not invent components. Only use component ids from the input.
6. Good reasons include API consumption, canonical data/model contract usage, result-object dependencies, configuration/schema ownership, orchestration callbacks, or backend service usage.
7. Bad reasons include topical similarity, shared vocabulary, belonging to the same subsystem, or likely future reuse.

RETURN JSON ONLY WITH THIS SCHEMA

```
{
  "cross_requirement_edges": [...],
  "same_requirement_edges": [...],
  "uncertain_edges": [...],
}
```

```
"summary": {"notes": "optional short note"}
}
INPUT: {prompt_payload}
```

Structural Action Selection Prompt

You are a software project strategist. Given an architecture design, determine the optimal repository actions.

```
Requirement: {requirement_info.name}
Architecture Components: {components_desc}
```

AVAILABLE ACTIONS

`save`, `split`, `revise`, `merge`, `add`, and `delete`.

ACTION SEMANTICS

- Treat `save` as the default when the current component boundary and responsibilities are already structurally appropriate.
- Treat `split` as a structural refinement signal first, not just a response to component size.
- Do not use `split` to create one module per feature; prefer shared module families for nearby features.
- Treat `revise` as a boundary-preserving revision that updates responsibilities or implementation strategy without creating a new component.
- Use `add` primarily during initial structure establishment when a sub-requirement still lacks a concretely realized component; later rounds should prefer `save`, `split`, `revise`, `merge`, or `delete` when revising existing structure.
- Preserve requirement vocabulary so later planning can retain domain families.
- In multi-round feedback, consider previous feedback for each component and revise actions only when needed.

OUTPUT FORMAT

Return only a JSON array in the form [{ "component": "...", "action": "...", "rationale": "..." }].

New Requirement Integration Prompt

Context

You are a strict Requirements Analyst. Your goal is to integrate a New Requirement into a DAG.

Input Data

```
New Requirement: {new_requirement}
Existing Requirements Candidates:
{existing_reqs_text}
```

Decision Logic (Follow Strictly)

- Step 1: Check for duplicates. If the new requirement has exactly the same semantic meaning, tag it as `EXISTING`.
- Step 2: Check for independence (the “`ADD`” test). Ask whether the new requirement can exist or be implemented even if the candidate requirement is deleted. Only if it is clearly independent and not a subset of an existing requirement should it be tagged as `ADD`.
- Step 3: If Step 2 fails, check for a strict relation. Use `CHILD`, `MERGE`, `SPLIT`, or `DELETE` only when there is a real dependency, refinement, or conflict.

CRITICAL CONSTRAINTS

- Do not create a relation just because two requirements share keywords.
- Link them only when there is a clear functional dependency or strong design coupling.
- Balance `ADD` and `RELATION`; avoid overusing either tag.

OUTPUT FORMAT (JSON ONLY)

```
{
  "tag": "EXISTING|ADD|RELATION",
  "relation_type":
    "CHILD|MERGE|SPLIT|DELETE|null",
  "affected_requirements": ["name_string"],
  "reason": "...",
  "confidence": 0.0
}
```

```
}
```

Requirement-Level DAG Evolution Prompt

You are analyzing how to integrate a new requirement into an existing requirement DAG.

```
New Requirement: {new_requirement}
Existing DAG Summary: {dag_summary}
Existing Requirements Sample:
{existing_requirements}
Context: {context}
```

ANALYZE THE IMPACT AND DECIDE THE BEST REQUIREMENT-INTEGRATION OPERATION

1. create/add: use when the requirement is genuinely new and does not overlap with existing nodes.
2. split: use when the new requirement shows that an existing node should be broken down.
3. merge: use when the new requirement overlaps substantially with existing nodes.
4. delete: use when the new requirement makes existing nodes obsolete.
5. save: use when the new requirement does not require any structural change to the existing nodes.

RETURN ONLY JSON

```
{"operation": "create|add|split|merge|delete|save",
 "reason": "...",
 "confidence": 0.0,
 "target": "existing_requirement_name",
 "targets": ["req_a", "req_b"],
 "suggested_parents": ["parent_requirement"]}
```

Metric-Guided Component Splitting Rule

Component splitting is deterministic and is not delegated to an LLM. A component becomes eligible only when the metric-action stage marks it with `recommended_action = split` because its cohesion is below the split threshold and it serves enough sub-requirements in the current round.

```
Parent requirement: {parent_task}
Source component payload: {source_component}
```

PROCEDURE

1. Read the ordered, deduplicated `source_component.serves_subrequirements` set. If it contains fewer than two items, keep the component unchanged.
2. Set the number of output components to `min(split_max_output_components, max(2, min(3, |S(c)|)))`.
3. Partition the served sub-requirements deterministically into this many groups using stable round-robin assignment.
4. Create one child component per group, named from the source component with a stable part suffix. Each child keeps the source responsibilities, or receives requirement-derived fallback responsibilities when the source has none.
5. Validate that every child has a non-empty name, non-empty responsibilities, and served sub-requirements drawn only from the source component. Accept the split only if at least two valid children remain.
6. Accepted splits use confidence 1.0 and the reason `Deterministic metric split accepted without LLM approval`.

OUTPUT RECORD

```
{"decision": "keep|split",
 "reason": "...",
 "confidence": 1.0,
 "split_into": ["child_component_name"]
}
```

Metric Merge Judge Prompt

You are a strict software architecture merge reviewer. The metric stage has already proposed one merge candidate from component coupling; your task is only to decide whether the candidate is semantically safe.

Input JSON: {metric_merge_candidate}

COUPLING DEFINITION

The candidate score is `Jaccard(served_subrequirements) * layering_penalty`. The Jaccard term measures overlap between the two served-subrequirement sets. The layering penalty discounts pairs whose names suggest complementary layers such as `core/API`, `engine/integration`, or `core/adaptor`.

APPROVAL RULES

1. Treat high coupling as evidence of similar requirement scope, not as proof that merging is safe.
2. Approve only if the smaller component is mostly redundant and both components have the same core responsibility.
3. Reject merges that would collapse a clean `API/engine`, `parser/builder`, `core/adaptor`, or other layered separation.
4. Reject when there is an interface conflict, behavior conflict, or high architectural risk.
5. Do not merge any component whose recommended action is `save` or `revise`; those actions mean the boundary should remain stable for the round.

RETURN JSON ONLY

```
{"approved": true,
 "same_responsibility": true,
 "interface_conflict": false,
 "behavior_conflict": false,
 "risk": "low",
 "reason": "short reason"}
```

E. Code Generation and Repair

Repository materialization first plans file layout, then follows a skeleton–test–fill workflow, after which repair prompts address import, syntax, and lint failures.

File Layout Planning Prompt

You are planning file layout for a Python project.

```
Requirement: {requirement_name}
Description: {requirement_description}
Components: {component_list}
```

RULES

1. Output one file path for each component.
2. File paths must be relative and end with `.py`.
3. Keep paths under the allowed roots when possible.
4. Prefer stable package layout and avoid creating new top-level roots.
5. Preserve domain semantics from the requirement and served sub-requirements.
6. Avoid collapsing unrelated domain families into generic buckets such as `core`, `common`, `runtime`, `api`, or `services` unless a domain-specific nested package is present.
7. When a generic package is unavoidable, prefer `core/<domain_family>/module.py` over `core/module.py`.
8. Do not create one subpackage per named feature by default; nearby features should share the same stable module family.

RETURN ONLY JSON

```
{"plans": [{"component_name": "ComponentName",
 "file_path": "package/subpkg/module.py"}]}
```

Skeleton Generation Prompt

You are an expert Python engineer. Produce a SKELETON module only (phase one of two).

```
=== PROJECT REQUIREMENT ===
Name: {req_name}
Description: {req_description}
Dependencies: {req_dependencies}
=== ARCHITECTURE DESIGN ===
Rationale: {architecture_rationale}
=== PARENT CONTEXT ===
Parent Node: {parent_node}
=== COMPONENT ===
Name: {component_name}
Responsibilities: {responsibilities}
=== RELATED COMPONENTS === {context_components}
```

RULES FOR SKELETON

1. Include imports, module docstring, and public classes/functions with full signatures and docstrings.
2. Use `raise NotImplementedError("TDD")` only for concrete members; do not use `pass` or `...` as placeholders in public concrete members.
3. Preserve names and shapes implied by responsibilities; this API will be tested in phase two.
4. Reuse names and import paths from implemented components exactly when needed.
5. Every public declaration must be valid Python syntax and use legal Python identifiers.
6. Core algorithms must be designed and implemented inside this generated repository.

RETURN ONLY JSON

```
{"file_path": "<path>.py", "code": "<full skeleton source>"}
```

Pytest Generation Prompt

You write pytest for the following skeleton. Repo root will be on PYTHONPATH.

```
Module relative path: {resolved_file_path}
Import module as: {module_q}
Component: {component_name}
Responsibilities: {responsibilities}
Skeleton source: {skeleton_code}
```

RULES

1. Use pytest style.
2. Tests may initially fail because the skeleton is not implemented yet, but do not lock in `NotImplementedError` behavior for concrete APIs.
3. Only expect `NotImplementedError` for explicitly abstract interfaces.
4. Cover the final public API implied by the skeleton and responsibilities, including return shapes, state changes, and integration behavior where possible.
5. `test_file_path` should be under `tests/` and use a stable name.
6. Treat the reviewed skeleton interfaces as the source of truth; do not invent alternate names.

RETURN ONLY JSON

```
{"test_file_path": "tests/test_...py",
 "test_code": "<full pytest file source>"}
```

Full Implementation Prompt

You now FULLY implement the module so that the provided pytest passes.

```
Module file path: {resolved_file_path}
```

```
Import root qualification: {module_q}
Reviewed skeleton: {skeleton_code}
Pytest file: {test_code}
Implemented components context:
{implemented_components_context}
```

RULES

1. Return complete module source in key `code` only.
2. Remove every concrete placeholder such as `NotImplementedError("TDD")`, `pass`, or `...`
3. Implement according to the reviewed skeleton's public API, docstrings, and stated semantics.
4. Match imports and names from the skeleton exactly.
5. Match import paths and symbol names from implemented components exactly; do not invent new names.
6. If a dependency API is ambiguous, add a small local compatibility wrapper in this file rather than importing a symbol that does not exist.
7. The final module must compile as valid Python.
8. Implement the component's core algorithm locally in this repository or by reusing already generated local modules.

RETURN ONLY JSON

```
{"code": "<full implemented source>"}
```

Lint Patch Repair Prompt

You are fixing Python code to satisfy lint/syntax checks.

```
Attempt: {attempt_no}
Target file path for patch: {virtual_path}
Lint diagnostics: {diagnostics}
Current code: {content}
```

HARD REQUIREMENTS

1. Return only a unified diff patch for this single file.
2. The patch must only touch the target file.
3. Use headers exactly `--- a/<path>` and `+++ b/<path>`.
4. Include valid `@@` hunk lines.
5. Do not include markdown fences or extra text.

Lint Full-File Fallback Prompt

You are fixing Python code to satisfy lint/syntax checks.

```
Attempt: {attempt_no} (full-file fallback mode)
File path: {file_path}
Lint diagnostics: {diagnostics}
Current code: {content}
```

REQUIREMENTS

1. Return only corrected Python code.
2. Keep behavior the same unless required to fix errors.
3. Do not add markdown fences.

Import Postcheck Repair Prompt

Repair a Python import smoke failure for component `{component_name}`.

```
Target module: {module_name}
Planned implementation path: {planned_file_path}
Import failure trace: {failure_trace}
```

GOAL

Make `importlib.import_module(target_module)` succeed.

CONSTRAINTS

1. Fix the import-time or runtime error shown in the trace.
2. Keep file paths unchanged.
3. Preserve existing public APIs and exported symbol names unless the stack trace proves they are wrong.

4. Prefer minimal repairs to top-level initialization, registry or state shape mismatches, and bad imports.
5. Do not remove required behavior just to silence the import error.
6. Do not introduce placeholder code or `NotImplementedError`.